Глава 6.
Коллекции языка C#

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Дисциплина Алгоритмизация и программирование
Лектор: д.т.н., проф. Иванова Галина Сергеевна

6.1 Основные коллекции .NET

Коллекция - это группа связанных объектов данных, хранящихся в спец. структуре. В отличии от массива коллекция всегда одномерна и не фиксирована по длине. Различают:

- *необобщенные коллекции* – все элементы типа Object – определены в пространстве System.Collections;
- *обобщенные коллекции* – все элементы одного (заданного типа) – определены в пространстве System.Collections.Generic;
- *специальные коллекции* – оптимизированы для работы с данными конкретного типа – определены в пространстве System.Collections.Specialized

Коллекция	**Необобщенный класс**	**Обобщенный класс**
Список	ArrayList	List<T>
Стек	Stack	Stack<T>
Очередь	Queue	Queue<T>
Множество	-	HashSet<T>
Ассоциативный массив, хранящий пары ключ-значение.	Hashtable (упорядочена по хеш-кодам ключей) SortedList (упорядочена по ключам)	Dictionary <TKey, TValue>

7.2 Интерфейсы

Все коллекции реализуют специальные интерфейсы.

Интерфейс – особый абстрактный класс, описывающий ряд общих свойств и методов (сигнатуры), которые должны быть предоставлены любым классом, реализующим интерфейс. В отличие от абстрактных классов интерфейсы могут реализовываться *различными иерархиями классов*. Класс может реализовывать несколько интерфейсов.

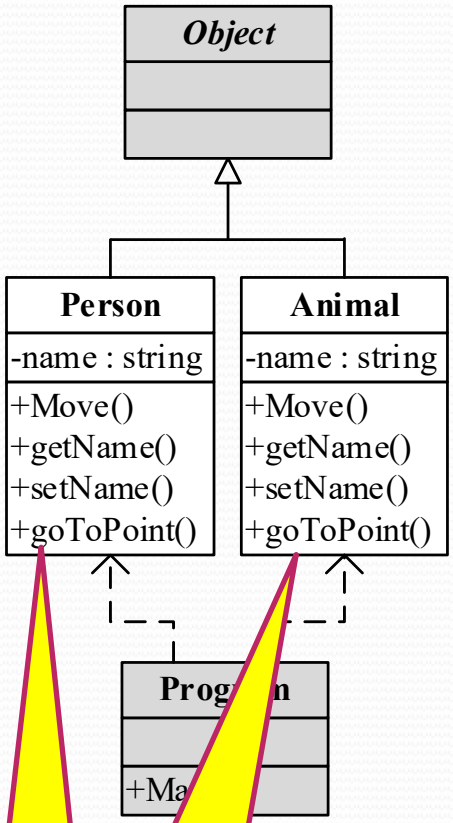
Синтаксис интерфейса:
[**Атрибуты**][**Спецификаторы**] **`interface`** **Имя**[; **Предки**] **Тело**

Спецификаторы: **`new`** (для вложенных интерфейсов), **`public`**, **`protected`**, **`internal`** (по умолчанию), **`private`**.

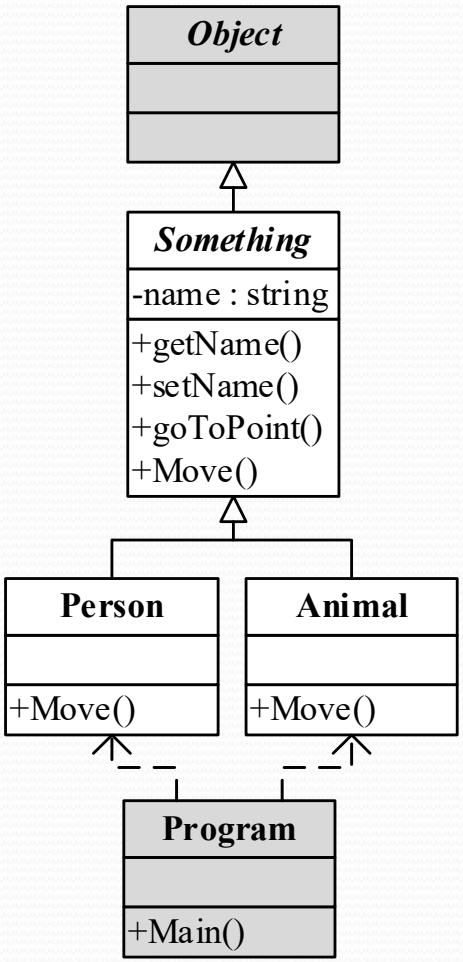
Все методы и свойства по умолчанию **`public`** и не могут иметь других спецификаторов. Методы в реализующем интерфейс классе также должны иметь спецификатор **`public`**. Интерфейсы могут наследоваться.

Пример использования интерфейса

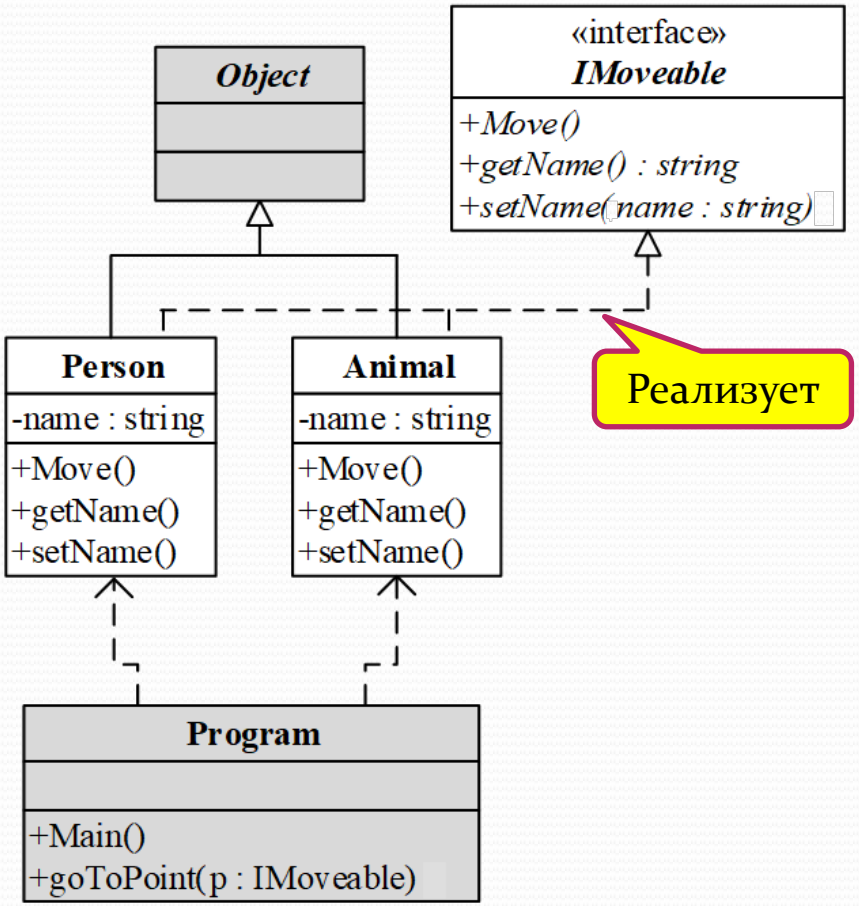
Ex6_01



Кроме Move() методы идентичны по записи



Решение с абстрактным классом



Решение с интерфейсом

Пример (продолжение)

```
using System;
namespace Ex6_01
{
    public interface IMoveable
    {
        string Name { get; set; }
        void Move();
    }
    public class Person : IMoveable
    {
        public string Name { get; set; }
        public void Move()
        {
            Console.WriteLine("Person is moving...");
        }
    }
    public class Animal : IMoveable
    {
        public string Name { get; set; }
        public void Move()
        {
            Console.WriteLine("Animal is moving...");
        }
    }
```

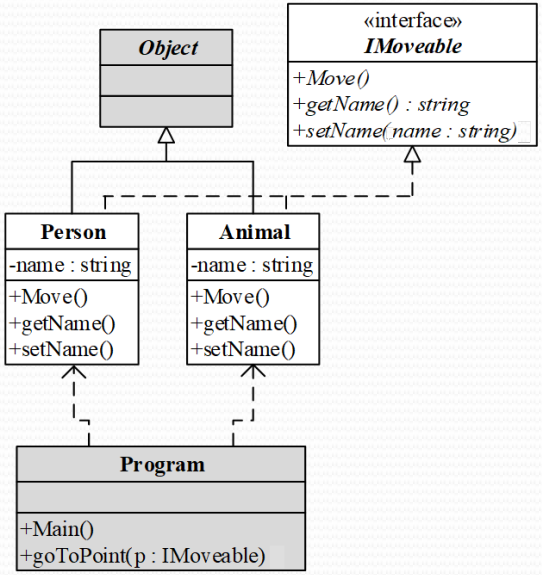
Пример (продолжение)

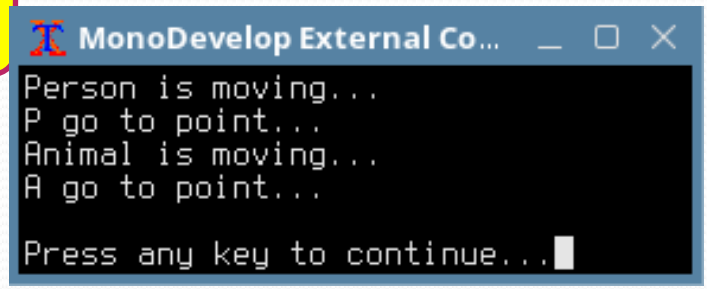
Указателю на объект базового класса можно присваивать адреса объектов производных...

```
class Program
    {
        static void GoToPoint(IMoveable m)
        {
            Console.WriteLine($"{m.Name} go to point...");
        }
        static void Main(string[] args)
        {
            Person p = new Person{ Name = "P" };
            p.Move();
            GoToPoint(p);

            IMoveable a = new Animal{ Name = "A" };
            a.Move();
            GoToPoint(a);
        }
    }
}
```

Создаем объект производного класса

Создаем объект базового класса-интерфейса

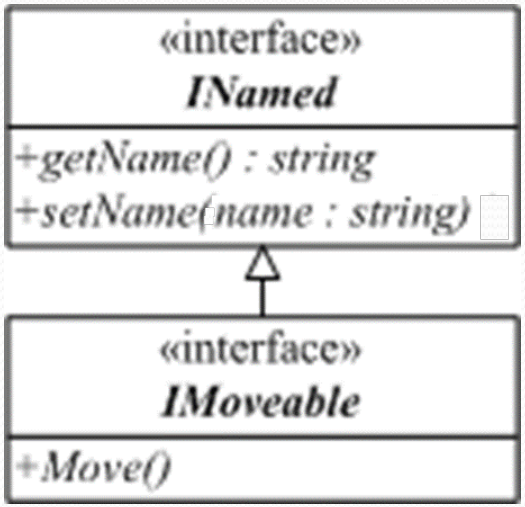




Наследование интерфейсов

Интерфейсы можно наследовать как классы:

```
public interface INamed
{
    string Name { get; set; }
}
public interface IMoveable: INamed
{
    void Move();
}
```



Проверка поддержки интерфейса объектом некоторого класса посредством операций as и is

Пусть

```
Person p = new Person{ Name = "P" };
```

Способ 1

Операция явного преобразования

```
IMovable im = p as IMoveable;
if (p is null) {…} // объект не реализует указанный интерфейс
else {…}           // поддерживает, можем что-то делать
```

Способ 2

Операция сопоставления с образцом

```
if (p is IMoveable) {…} // поддерживает, можем что-то делать
else {…}                // объект не реализует указанный интерфейс
```

Уточнение интерфейса при совпадении имен методов

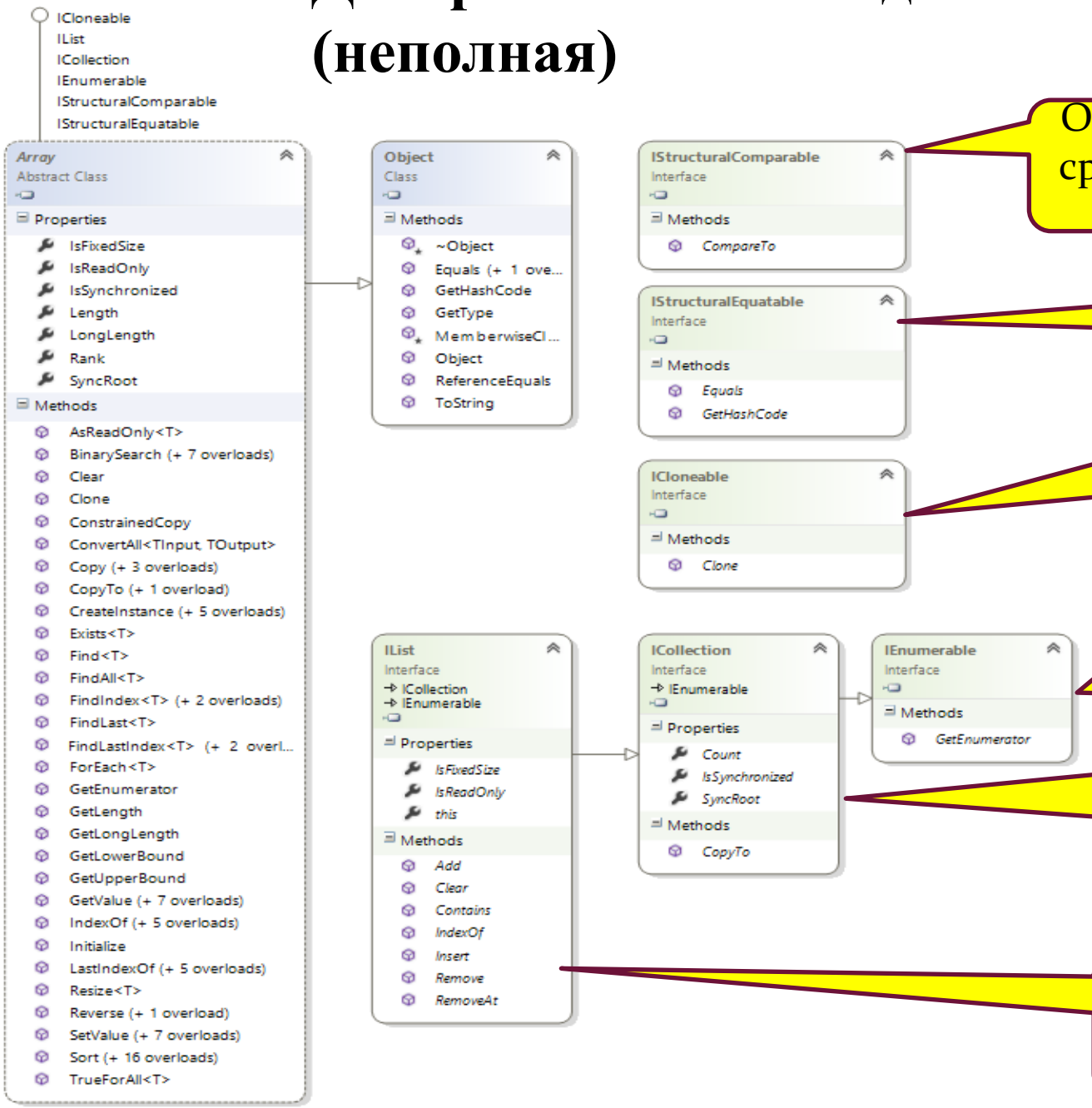
```
interface IMovable { void Move(); }
interface IMovable2 { void Move(); }

public class Person : IMovable, IMovable2
{
    public string Name { get; set; }
    void IMovable.Move()
        { Console.WriteLine("Move from iMovable1"); }
    void IMovable2.Move()
        { Console.WriteLine("Move from IMovable2"); }
}

Person p = new Person();
((IMovable)p).Move();
((IMovable2)p).Move();
```

Разные интерфейсы содержат методы с одинаковыми именами

Реализация обоих интерфейсов

Уточнение, метод какого именно интерфейса вызывается

Диаграмма классов для класса Array (неполная)



Определяет структурное сравнение объектов (для сортировки)

Определяет равенство объектов

Определяет клонирование объектов

Определяет перечислитель объектов для for и foreach

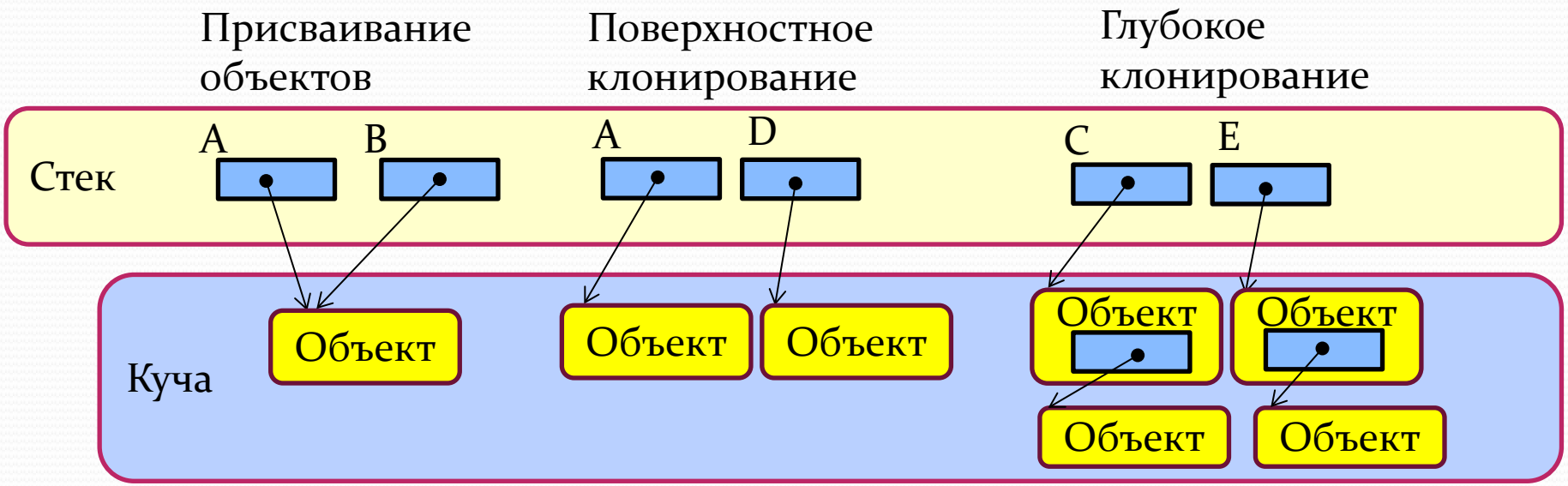
Определяет количество и копирование объектов

Определяет работу с последовательностью объектов

Интерфейс ICloneable

При выполнении присваивания объектов создается копия ссылки на объект. Если необходима независимая копия, то необходимо реализовывать интерфейс ICloneable, который определяет метод Clone(), осуществляющий копирования объекта:

- *поверхностное клонирование* - без выделения памяти под ссылочные переменные – применяется для объектов, не имеющих ссылочных полей;
- *глубокое клонирование* – с выделением памяти под ссылочные поля объекта.



Пример:

```
    Person A = new Person("Tom", 23);   // создание объекта
а)  Person B = A;                       // копирование ссылки
б)  Person D = (Person)A.Clone();       // клонирование объекта
```

Пример поверхностного клонирования

```
class Person : ICloneable
{
    public string Name { get; set; }
    public int Age { get; set; }
    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }
    public object Clone()
    {
        return new Person(Name, Age);
    }
}
```

Конструктор

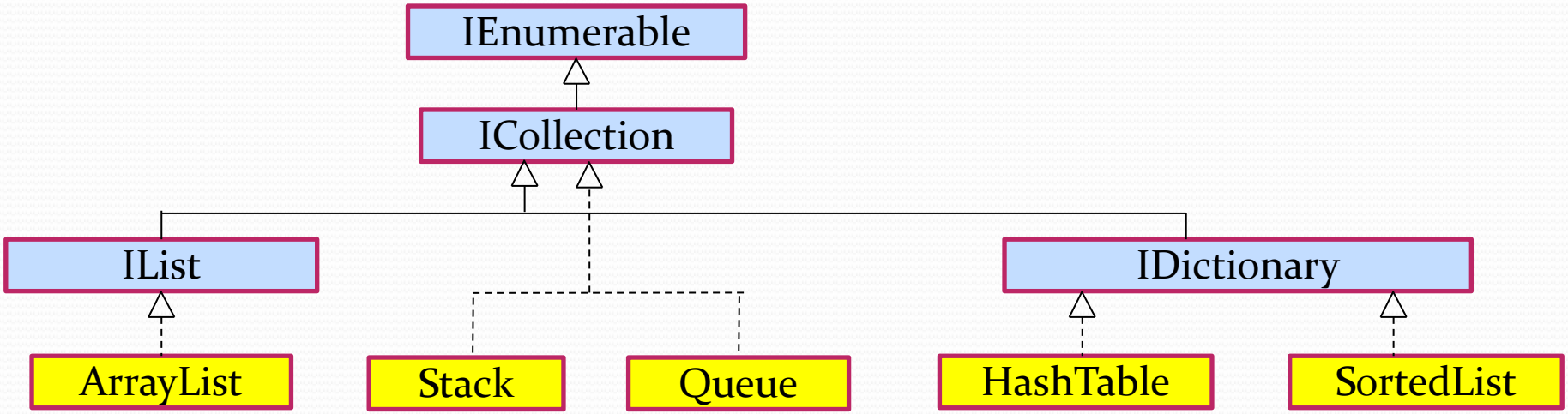
Метод клонирования

Создание объекта и его копии:

```
var Tom = new Person("Tom", 23);  // создание объекта
var Bob = (Person)Tom.Clone();    // клонирование объекта
Bob.Name = "Bob";                 // изменение поля объекта-клона
Console.WriteLine(Tom.Name);      // поле исходного объекта - Tom
```

6.3 Основные интерфейсы необобщенных коллекций

Фрагмент иерархии интерфейсов и коллекций



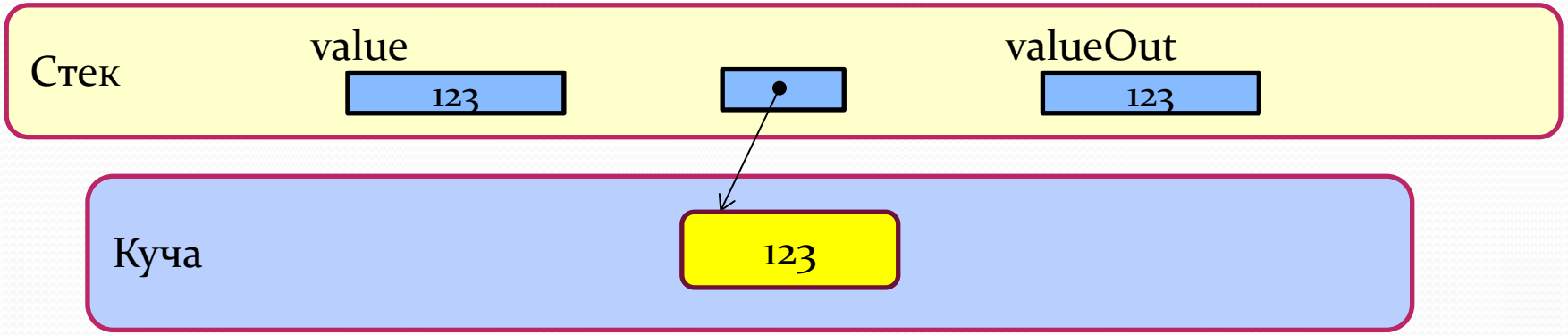
Интерфейс	Описание
IEnumerable	Реализует перечислитель, который используется операторами for и foreach.
ICollection	Объявляет свойство Count - количество элементов и метод CopyTo() - копирование в массив
IList	Обеспечивает доступ по индексу, а также методы вставки Insert() и удаления элементов Remove() по номеру позиции, очистки коллекции Clear() и т.п.
IDictionary	Для ассоциативных массивов. Определяет поведение коллекции, которая должна хранить объекты в виде пар ключ-значение

Упаковка и распаковка данных типов-значений

Поскольку необобщенные коллекции используются для хранения объектов классов, наследуемых от Object, добавление в коллекцию типов-значений предполагает автоматическую упаковку в объекты, а извлечение – распаковку в соответствующий тип с помощью явного переопределения типа.

Пример:

```
int value = 123;
object boxed = value;                // неявная упаковка int
int valueOut = (int)boxed;           // явная распаковка
Console.WriteLine(valueOut);         // вывод "123"
```



Интерфейс IEnumerable

Основной метод:

- **IEnumerator GetEnumerator ()** – возвращает объект-перечислитель, используемый для прохода по коллекции, используется for и foreach.

Интерфейс ICollection

Основное свойство:

- **Count** – количество элементов в коллекции.

Основной метод:

- **void CopyTo(Array array, int index)** - копирует коллекцию в массив array, начиная с указанного индекса.

Интерфейс IList

Основные свойства:

- **IsFixedSize** – фиксированное количество элементов в коллекции;
- **IsReadOnly** – только для чтения;
- **Item**[] – обращение к элементу по индексу.

Основные методы:

- **int Add(object? value)** - добавляет к коллекции объект value, возвращает индекс или -1;
- **void Remove(object value)** - удаляет из коллекции объект value;
- **void RemoveAt(int index)** - удаляет из списка элемент по индексу index;
- **void Clear()** - удаляет из коллекции все элементы;
- **bool Contains(object? value)** - проверяет, содержится ли в коллекции объект value. Если содержится, возвращает true, иначе возвращает false;
- **int IndexOf (object? value)** – определяет индекс элемента по его значению или возвращает -1;
- **void Insert(int index, object? value)** – вставляет элемент в указанное место.

6.3.1 Коллекция ArrayList (реализует IList)

Массив разнотипных объектов классов, наследуемых от Object:

ArrayList Имя = new ArrayList ([Размер]) [{Список_значений}];

Размер - определяет начальный размер коллекции, по мере заполнения автоматически увеличивается (обычно вдвое).

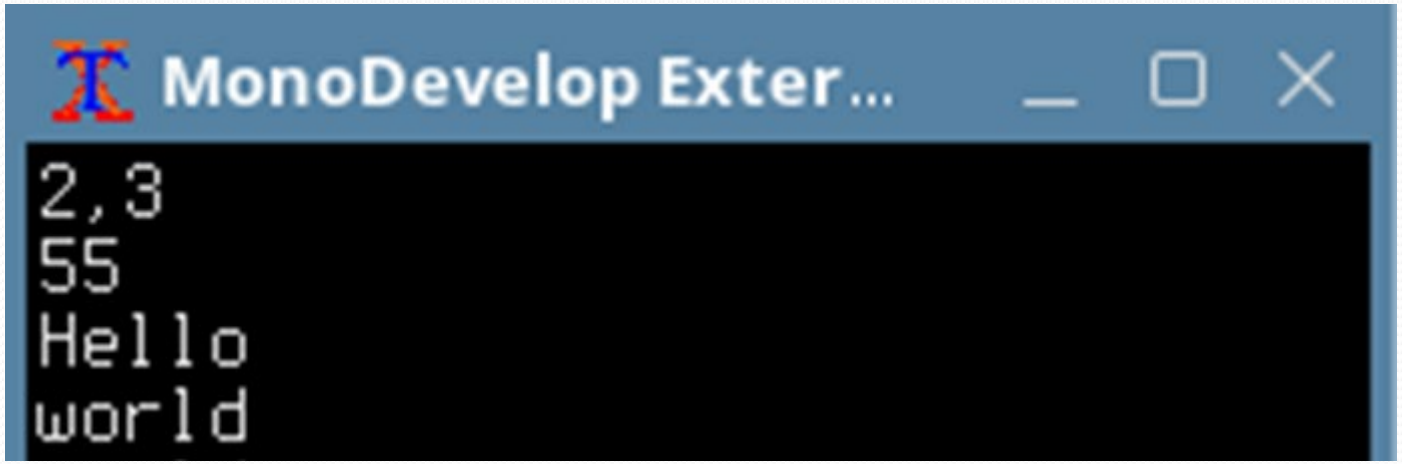
Специфические методы:

- **int LastIndexOf(object value)** - возвращает индекс последнего вхождения в массив объекта value;
- **void AddRange(ICollection col)** - добавляет объекты коллекции col;
- **ArrayList GetRange(int index, int count)** - возвращает новый массив - фрагмент, который содержит count элементов, начиная с индекса index;
- **void InsertRange(int index, ICollection col)** - вставляет в массив, начиная с индекса index, коллекцию ICollection;
- **void RemoveRange(int index, int count)** - удаляет из массива count элементов, начиная с индекса index;
- **void Reverse()** - переворачивает массив.

Пример

Ex6.02

```
ArrayList list = new ArrayList();
list.Add(2.3);      // занесение в массив объекта типа double
list.Add(55);       // занесение в массив объект типа int
list.AddRange(new string[] {"Hello","world"});
foreach (object o in list)     // перебор значений
    Console.WriteLine(o);
```



Пример (2)

```
list.RemoveAt(0);                    // удаление первого элемента
list.Reverse();                      // переворот массива
Console.WriteLine(list[0]);  // получение элемента по
                                     // индексу
for (int i = 0;i < list.Count;i++)  // перебор
                                     // значений
    Console.WriteLine(list[i]);
Console.WriteLine(list.Contains(55));
```

```
world
world
Hello
55
True
```

Пример (окончание)

```
foreach (object o in list) {
    string type = o.GetType().Name; //возвращает тип объекта
    if (type == "Int32")
        Console.WriteLine("Целое: " + o.ToString());
    else
        if (type == "String")
          Console.WriteLine("Строка: " + o.ToString());
        else
          Console.WriteLine("Другой : " + o.ToString());
}
```

```
Строка: world
Строка: Hello
Целое: 55

Press any key to continue...
```

6.3.2 Коллекция Hashtable

Объявление ассоциативной коллекции:

Hashtable Имя = new Hashtable([Размер]) [{Список_пар_значений}];

Все ключи должны быть уникальны. При хранении ключи хешируются в целые числа, и данные хранятся по соответствующим индексам, следовательно, ключ можно использовать в качестве индекса!

Каждый элемент коллекции - это объект DictionaryEntry с двумя свойствами: объектом key и объектом value.

Специфические свойства:

- **ICollecrion Keys** - коллекция ключей таблицы
- **ICollecrion Values** - коллекция значений таблицы

Специфические методы (основные):

- **bool ContainsKey(Object key)** - проверка наличия ключа
- **bool ContainsValue(Object value)** - проверка наличия значения

Пример использования ассоциативной коллекции

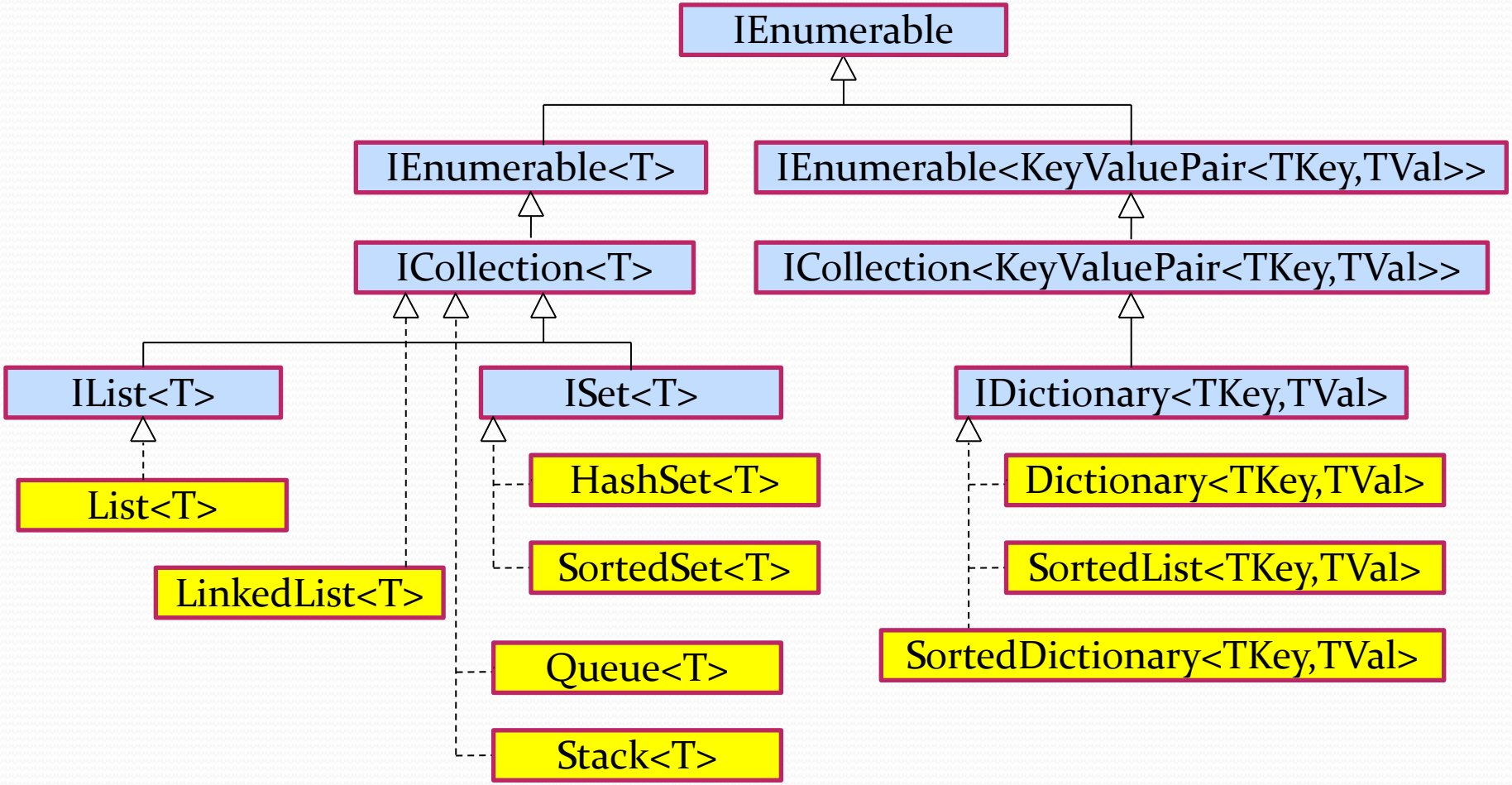
Ex6.03

```
Hashtable users = new Hashtable();
users.Add("andyb", "Andrew Brown");
users.Add("alexg", "Alex Green");
users["adrien"] = "Adrienne Black"; // добавление по индексу
Console.WriteLine(users.Count);
Console.WriteLine("{0}",users.ContainsKey("alexq"));
foreach(DictionaryEntry de in users)
  Console.WriteLine("{0} \t: {1}", de.Key, de.Value);
ICollection keys = users.Keys;      // извлечение ключей
foreach(object key in users.Keys)
  Console.WriteLine("{0} {1},key,users[key]);
```

```
3
False
alexg    : Alex Green
andyb    : Andrew Brown
adrien   : Adrienne Black
alexg Alex Green
andyb Andrew Brown
adrien Adrienne Black
```

6.4 Основные интерфейсы обобщенных коллекций

Фрагмент иерархии интерфейсов и коллекций



Примечание. На диаграмме не показан интерфейс ICompare<T>, на базе которого реализуется сравнение элементов для сортированных коллекций.

Основные коллекции

Интерфейс	Описание
`List<T>`	«Список», построен на базе динамического массива
`SortedList<T>`	Упорядоченный «список», реализован как двоичное сортированное дерево
`Linkedlist<T>`	Двусвязный список
`Stack<T>`	Стек
`Queue<T>`	Очередь
`HashSet<T>`	Неупорядоченное множество
`SortedSet<T>`	Упорядоченное множество
`Dictionary<K,V>`	Ассоциативный массив
`SortedList<K,V>`	Ассоциативный массив, упорядочен по ключу, реализован как сортированное двоичное дерево
`SortedDictionary<K,V>`	Ассоциативный массив, упорядочен по ключу

Описание интерфейсов обобщенных коллекций

Интерфейс	Описание
IEnumerable<T>	Реализует перечислитель для оператора foreach.
ICollection<T>	Объявляет свойство Count - количество элементов и методы CopyTo() - копирование в массив, Add(), Remove(), Clear(), Contains().
IList<T>	Для массивов. Доступ по позициям Item[], а также методы вставки Insert() и удаления элементов Remove() по номеру.
ISet<T>	Для множеств. Позволяет комбинировать различные элементы в объединения, а также проверять пересечение двух множеств.
IDictionary<K,V>	Для ассоциативных массивов. Доступ по ключам и значениям.
ILookup<K,V>	Обеспечивает работу с множественными значениями для одного ключа.
ICompableTo<T>	Используется для сравнения элементов коллекции с помощью метода CompareTo<T>().
IEqualityComparer<T>	Используется при сравнении ключей словаря.

Вычислительная сложность основных операций

Коллекция	Обращение по индексу	Добавление элемента	Удаление элемента	Поиск
`List<T>`	O(1)	O(1) – в конец O(n) – в начало	O(1) – последнего O(n) - первого	O(1) – по индексу O(n) - по значению
`Linkedlist<T>`	-	O(1) без поиска	O(1) без поиска	O(n)
`Stack<T>`	-	O(1)	O(1)	-
`Queue<T>`	-	O(1)	O(1)	-
`SortedSet<T>`	-	$O(\log_2 n)$	$O(\log_2 n)$	$O(\log_2 n)$
`Dictionary <K,V>`	O(1)	O(1) – в лучшем O(n) – в худшем	O(1)	O(1)
`SortedList <K,V>`	O(1)	O(n)	O(n)	$O(\log_2 n)$
`SortedDictionary <K,V>`	O(1)	$O(\log_2 n)$	$O(\log_2 n)$	$O(\log_2 n)$

6.4.1 Коллекция List<T>

Реализована на базе динамического массива. Объявление списка:

```
List<Тип> Имя =
                new List<Тип>([Размер])[{Список_значений}];
```

Свойства:

- **int Count** – количество элементов;
- **int Capacity** – количество элементов, которое может вместить список без изменения размера.
- **T Item[int index]** – возвращает или задает элемент по индексу.

Методы, реализующие интерфейс ICollection<T>:

- **void CopyTo(T[] array)** - копирует список в массив array;
- **void Add(T item)** - добавляет элемент в список;
- **bool Remove(T item)** - удаляет элемент item из списка, и если удаление прошло успешно, то возвращает true. Если в списке несколько одинаковых элементов, то удаляется только первый из них;
- **void Clear()** – удаляет все элементы;
- **bool Contains(T item)** - возвращает true, если элемент item есть в списке.

Методы List<T>

Методы, реализующие интерфейс IList<T>:

- **void Insert(int index, T item)** - вставляет элемент item в список по index. Если такого индекса в списке нет, то генерируется исключение;
- **void RemoveAt(int index)** - удаляет элемент по указанному индексу index. Если такого индекса в списке нет, то генерируется исключение.

Специфические методы:

- **void CopyTo(int index, T[] array, int arrayIndex, int count)** - копирует count элементов, начиная с index в массив array начиная с arrayIndex;
- **void AddRange(IEnumerable<T> collection)** - добавляет в список коллекцию или массив;
- **void InsertRange(int index, collection)** - вставляет коллекцию collection в список, начиная с индекса index. Если такого индекса в списке нет, то генерируется исключение;
- **List<T> GetRange(int index, int count)** - возвращает список элементов, количество которых равно count, начиная с индекса index;

Методы List<T> (2)

- **int BinarySearch(T item)** – осуществляет бинарный поиск элемента в сортированном списке;
- **int IndexOf(T item)** - возвращает индекс первого вхождения элемента в список;
- **int LastIndexOf(T item)** - возвращает индекс последнего вхождения элемента в список;
- **void Reverse()** - изменяет порядок элементов на обратный;
- **void Reverse(int index, int count)** - изменяет порядок на обратный для элементов, количество которых равно count, начиная с индекса index;
- **void Sort()** - сортирует список, используя функцию сравнения по умолчанию;
- **void Sort(IComparer<T>? comparer)** - сортирует список с помощью объекта comparer интерфейса сравнения;
- **void Sort (Comparison<T> comparison)** – сортирует список, используя для сравнения делегат int Comparison<T>(T x, T y);

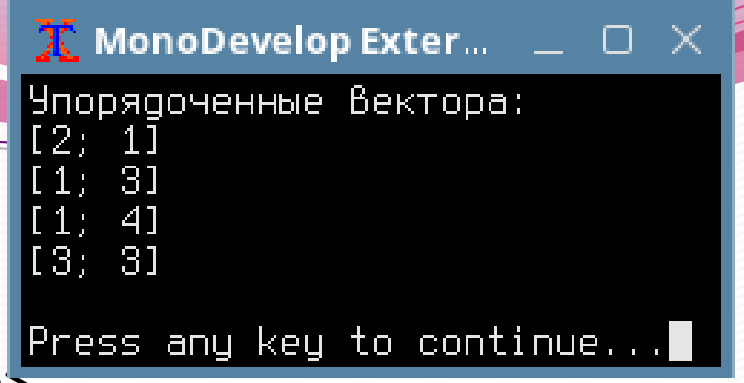
Пример

Ex6.04

```
using System;
using System.Collections.Generic;
namespace Ex6_04
{
    public class Point : IComparable<Point>
    {
        public int X { get; }
        public int Y { get; }
        public Point(int x, int y)
        {   X = x; Y = y;}
        public int CompareTo(Point other)
        {
            return Math.Sign(Math.Sqrt(X*X+Y*Y) -
            Math.Sqrt(other.X*other.X+other.Y*other.Y));
        }
        public override string ToString()
        {  return $"[{X}; {Y}]"; }
    }
```

Пример (окончание)

```
class Program
    {
        static void Main() {
            var points = new List<Point>
            {   new Point(3,3),new Point(1,3) };
            if (points[0].CompareTo(points[1]) > 0)
            {
                var tempPoint = points[0];
                points[0] = points[1];
                points[1] = tempPoint;
            }
            var a = new Point[]{new Point(1,4), new Point(2,1) };
             points.AddRange(a); // добавляет массив объектов Point[]
             points.Sort();      // вызывает метод CompareTo<Point>
             Console.WriteLine("Упорядоченные вектора: ");
             foreach (var point in points)
                 Console.WriteLine(point);
        }
    }
}
```



6.4.2 Коллекция SortedDictionary<K,V>

Создание коллекции:

```
SortedDictionary<Тип> Имя = new
      SortedDictionary<Тип>([Размер])[{Список_значений}];
```

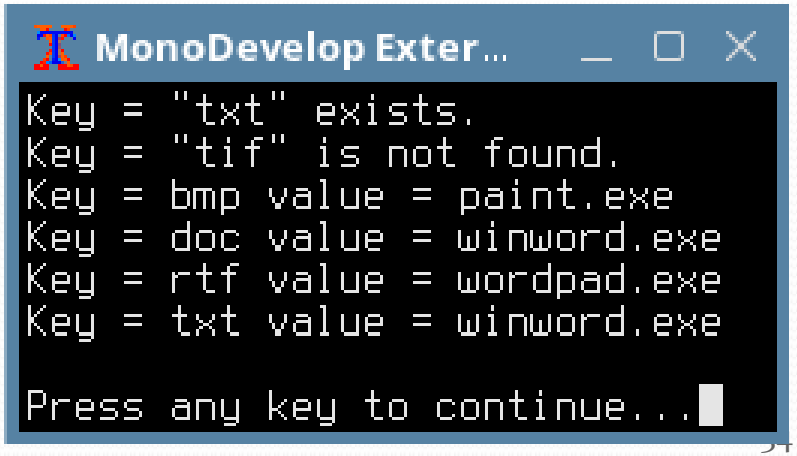
Методы:

- **void CopyTo(T[] array, int index)** - копирует список в массив структур array, начиная с указанного индекса;
- **void Add(T item)** - добавляет новую пару (элемент) в список;
- **bool Remove(T item)** - удаляет элемент item из списка, и если удаление прошло успешно, то возвращает true. Если в списке несколько одинаковых элементов, то удаляется только первый из них;
- **void Clear()** - удаляет из списка все элементы;
- **bool Contains(T item)** - возвращает true, если элемент item есть в списке.

Пример

Ex6.05

```
using System;
using System.Collections.Generic;
namespace Ex6_05
{
public class Program
{
   public static void Main()
   {
       SortedDictionary<string, string> openWith =
                 new SortedDictionary<string,string>();
       openWith.Add("txt", "notepad.exe");
       openWith.Add("bmp", "paint.exe");
       openWith.Add("rtf", "wordpad.exe");
       try  { openWith.Add("txt", "winword.exe");  }
       catch (ArgumentException)
          Console.WriteLine("Key = \"txt\" exists.");
```

Пример (окончание)

```
openWith["txt"] = "winword.exe"; // замена
openWith["doc"] = "winword.exe"; // добавление
try {
    Console.WriteLine("Key = \"tif\", value = {0}.",
                openWith["tif"]);// обращение к отсутствующему
}
catch (KeyNotFoundException)
{ Console.WriteLine("Key = \"tif\" is not found.");}
foreach(string key in openWith.Keys)
  Console.WriteLine("Key = {0} value = {1}",
        key,openWith[key]);
  }
}
}
```